

Assignment Eight

Hardware Lab: CS224

Shelke Durgesh Balkrishna: 230101093

Kavya Kumar Agrawal: 230101053

Parth Sunil Aher: 230101072

Dhruv Pansuriya: 230101071

22nd April 2025

1 Problem Statement

Implement a single-cycle 32-bit MIPS processor supporting a subset of instructions including:

- Memory reference instructions: **lw** (load word), **sw** (store word)
- Arithmetic-logical instructions: **add**, **sub**, **and**, **or**, **slt** (set-less-than)
- Control transfer instructions: **beq** (branch equal), **j** (jump)
- Instruction for supporting subroutine: **jal** (jump and link)
- Additional instruction implemented: **addi** (add immediate), **jr** (jump register)

The processor must execute one instruction per clock cycle (single-cycle implementation). All components of the datapath must be implemented as separate Verilog modules to maintain a modular design approach.

2 Instruction Format Summary

MIPS uses three instruction formats:

Format	Bit Fields	Example Instructions
R-type	op[31:26], rs[25:21], rt[20:16], rd[15:11], shamt[10:6], funct[5:0]	add, sub, and, or, slt, jr
I-type	op[31:26], rs[25:21], rt[20:16], immediate[15:0]	lw, sw, beq, addi
J-type	op[31:26], address[25:0]	j, jal

Table 1: MIPS Instruction Formats

2.1 OpCode and Function Field Summary

Instruction	Format	OpCode (hex)	Function (hex)
add	R-type	0x00	0x20
sub	R-type	0x00	0x22
and	R-type	0x00	0x24
or	R-type	0x00	0x25
slt	R-type	0x00	0x2A
jr	R-type	0x00	0x08 (custom)
lw	I-type	0x23	-
sw	I-type	0x2B	-
beq	I-type	0x04	-
addi	I-type	0x08	-
j	J-type	0x02	-
jal	J-type	0x03	-

Table 2: OpCode and Function Field Values

3 Datapath Design

The single-cycle MIPS processor executes each instruction in one clock cycle. The processor is designed with a modular approach, separating the datapath and control logic.

3.1 Overall Architecture

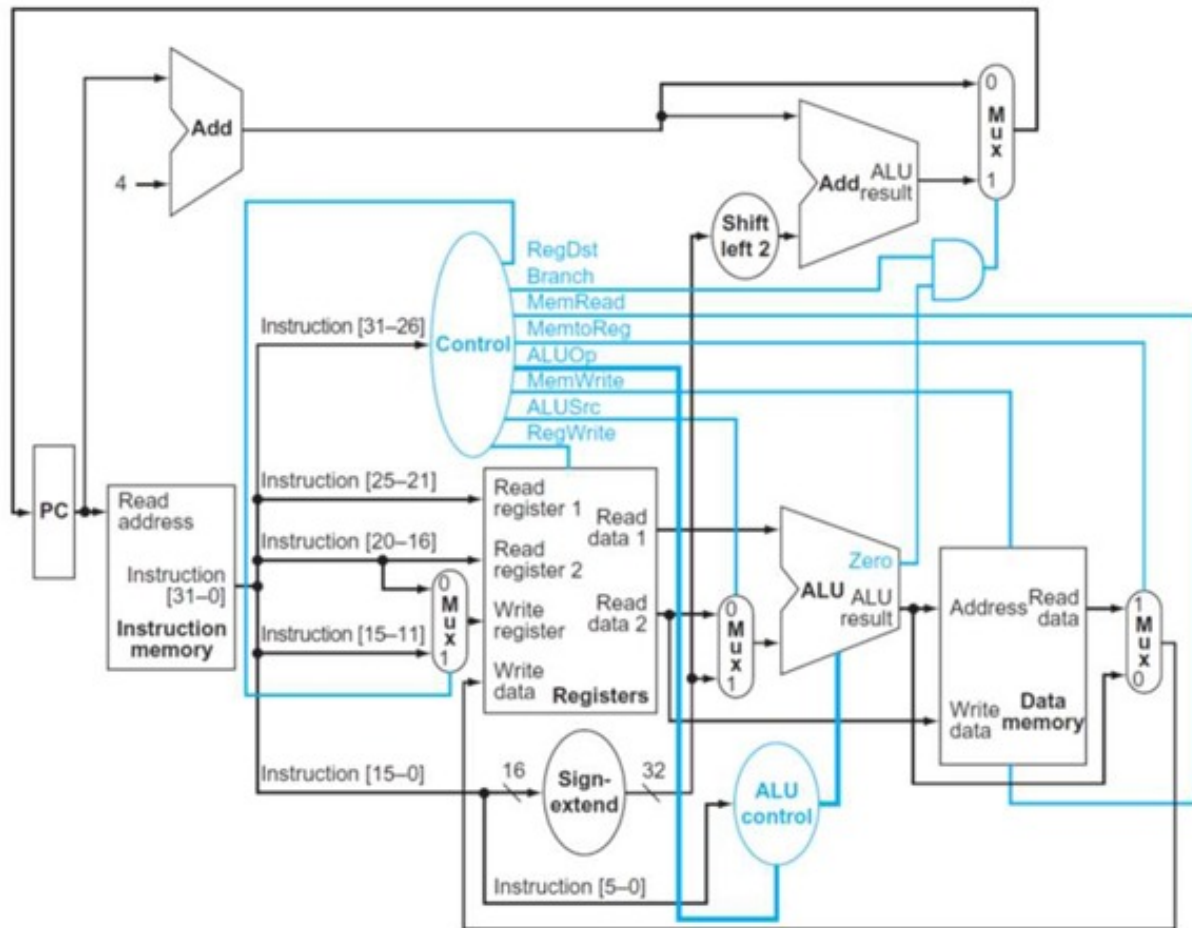


Figure 1: Single-Cycle MIPS Processor Datapath

Note: The above diagram doesn't have jal, but we have implemented it (it's working is explained further in the report)

3.2 Key Components

3.2.1 Program Counter (PC)

The PC module maintains the address of the current instruction. It has an enable input that allows updating the PC on each clock cycle when enabled. The PC is a 32-bit register with synchronous reset functionality.

3.2.2 Instruction Memory

The instruction memory module stores the program to be executed. It is implemented as a read-only memory with 256 32-bit locations. In our implementation, we initialize it with a test program that demonstrates all the supported instructions.

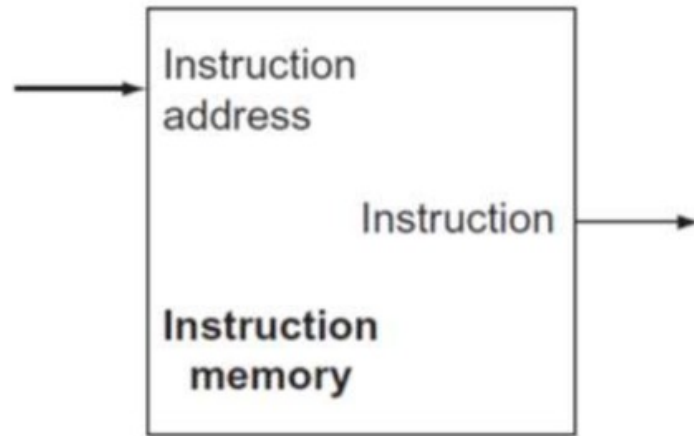


Figure 2: Instruction Memory

3.2.3 Register File

The register file consists of 32 general-purpose registers, each 32 bits wide. It has:

- Two read ports that allow reading two registers simultaneously
- One write port for writing data to a register
- Write enable signal to control when writing occurs
- Register \$0 is hardwired to zero

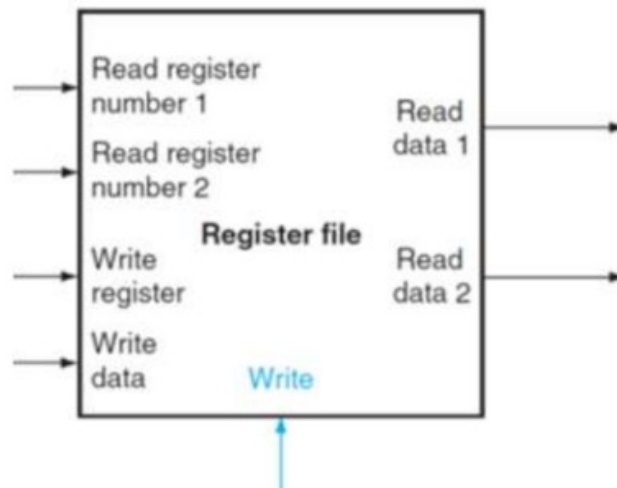


Figure 3: Register File

The implementation uses a decoder and multiplexers to select registers for reading and writing.

3.2.4 ALU (Arithmetic Logic Unit)

The ALU performs operations based on a 4-bit control signal:

- 0000: AND
- 0001: OR
- 0010: ADD
- 0110: SUB
- 0111: SLT (set if less than)

The ALU also outputs a zero flag that indicates whether the result is zero (used for branch decisions).

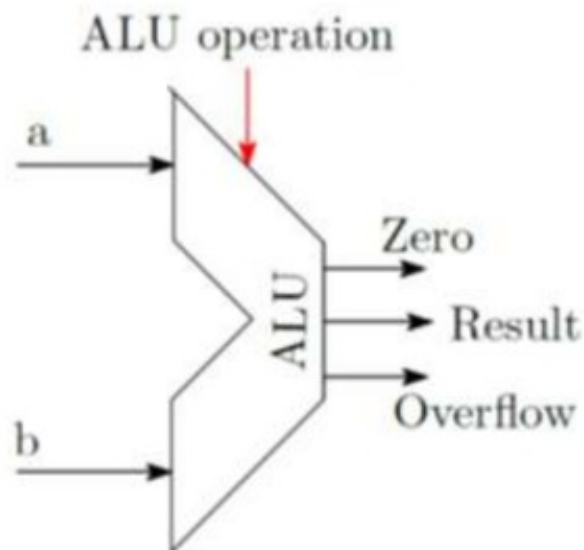


Figure 4: ALU

3.2.5 Data Memory

The data memory has 256 32-bit word locations, with separate read and write enable signals. Memory addresses are word-aligned (divided by 4).

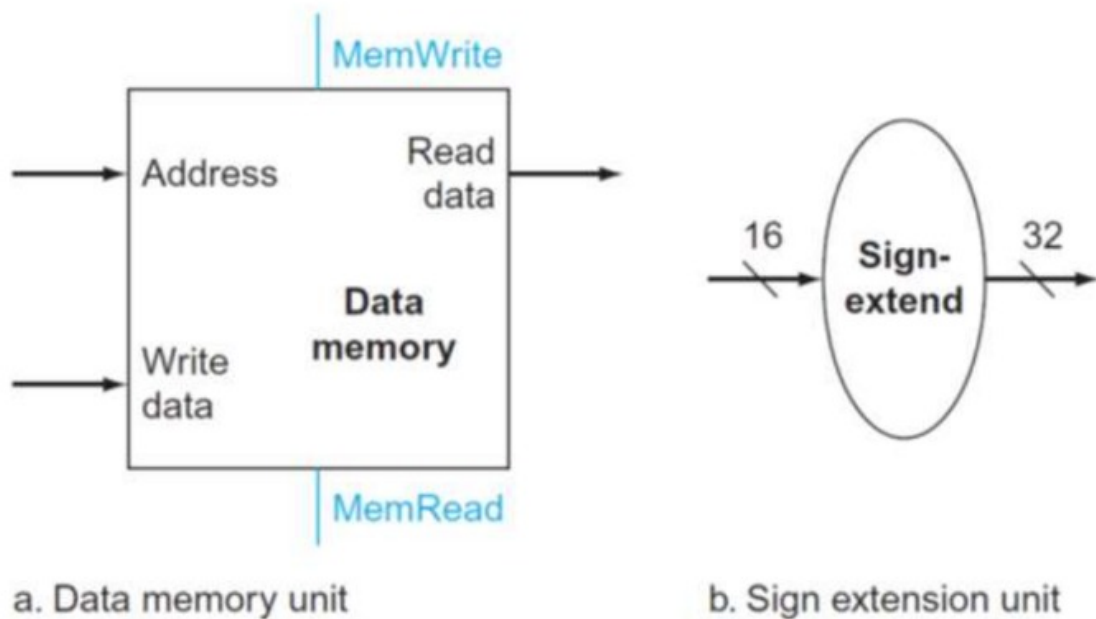


Figure 5: Data Memory

3.2.6 Sign Extender

Extends the 16-bit immediate values to 32 bits by replicating the most significant bit.

3.2.7 Shifters

Two types of shifters are used:

- Left shift by 2 bits (multiply by 4) for branch address calculation
- Left shift by 2 bits for jump address calculation

3.3 Data Path Operations

3.3.1 R-type Instructions (add, sub, and, or, slt)

1. Fetch instruction from instruction memory using PC
2. Read two source registers (rs, rt) from register file
3. Perform ALU operation based on function code
4. Write result to destination register (rd)
5. Increment PC by 4

3.3.2 Load Word (lw)

1. Fetch instruction from instruction memory
2. Read base register (rs) from register file
3. Sign-extend immediate field
4. Calculate memory address using ALU (base + offset)
5. Read data from memory at the calculated address
6. Write data to destination register (rt)
7. Increment PC by 4

3.3.3 Store Word (sw)

1. Fetch instruction from instruction memory
2. Read base register (rs) and data register (rt) from register file
3. Sign-extend immediate field
4. Calculate memory address using ALU (base + offset)
5. Write data to memory at the calculated address
6. Increment PC by 4

3.3.4 Branch Equal (beq)

1. Fetch instruction from instruction memory
2. Read two registers (rs, rt) from register file
3. Compare register values using ALU subtraction
4. Sign-extend and shift immediate field (multiply by 4)
5. If registers are equal (ALU zero flag is set):
 - Calculate target address ($PC + 4 + \text{shifted immediate}$)
 - Set PC to target address
6. Else: Increment PC by 4

3.3.5 Jump (j)

1. Fetch instruction from instruction memory
2. Shift jump target left by 2 bits
3. Combine upper 4 bits of $(PC + 4)$ with shifted target
4. Set PC to jump address

3.3.6 Jump and Link (jal)

1. Fetch instruction from instruction memory
2. Calculate return address $(PC + 4)$
3. Write return address to register \$31 (ra)
4. Shift jump target left by 2 bits
5. Combine upper 4 bits of $(PC + 4)$ with shifted target
6. Set PC to jump address

3.3.7 Jump Register (jr)

1. Fetch instruction from instruction memory
2. Read register (rs) from register file
3. Set PC to the value in rs

3.3.8 Add Immediate (addi)

1. Fetch instruction from instruction memory
2. Read source register (rs) from register file
3. Sign-extend immediate field
4. Add register value and sign-extended immediate
5. Write result to destination register (rt)
6. Increment PC by 4

4 Control Unit Design

The control unit is divided into two parts:

1. Main Controller: Generates control signals based on the OpCode
2. ALU Controller: Generates ALU control signals based on the ALUOp from the main controller and the function field for R-type instructions

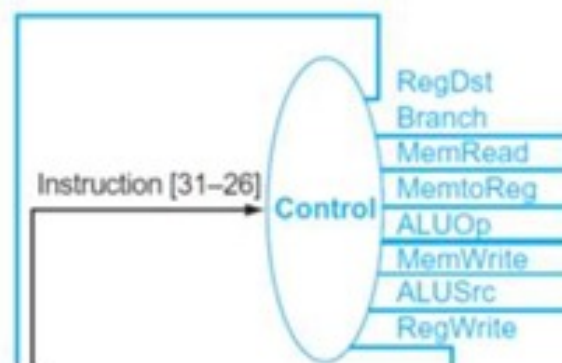


Figure 6: Controller

4.1 Main Controller

The main controller decodes the 6-bit OpCode and generates the following control signals:

Instr	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp	Jump	Link	JR
R-type	1	0	0	1	0	0	0	10	0	0	0
lw	0	1	1	1	1	0	0	00	0	0	0
sw	x	1	x	0	0	1	0	00	0	0	0
beq	x	0	x	0	0	0	1	01	0	0	0
j	x	x	x	0	0	0	0	xx	1	0	0
jal	x	x	x	1	0	0	0	xx	1	1	0
addi	0	1	0	1	0	0	0	00	0	0	0
jr	x	x	x	0	0	0	0	xx	0	0	1

Table 3: Control Signal Truth Table (Landscape Orientation)

- **RegDst**: Selects the destination register (rt or rd)
- **ALUSrc**: Selects the second ALU operand (rt or immediate)
- **MemtoReg**: Selects the data to write to the register file (ALU result or memory data)
- **RegWrite**: Enables writing to the register file
- **MemRead**: Enables reading from data memory
- **MemWrite**: Enables writing to data memory
- **Branch**: Indicates a branch instruction
- **ALUOp**: Determines the ALU operation (00 for add, 01 for subtract, 10 for R-type)
- **Jump**: Indicates a jump instruction
- **Link**: Indicates to save PC+4 in \$31 (for jal)
- **JR**: Indicates a jump register instruction

4.2 ALU Controller

The ALU controller takes the 2-bit ALUOp from the main controller and the 6-bit function field (for R-type instructions) to generate the 4-bit ALU control signal:

ALUOp	Function Field	ALU Control
00	x	0010 (ADD)
01	x	0110 (SUB)
10	100000 (add)	0010 (ADD)
10	100010 (sub)	0110 (SUB)
10	100100 (and)	0000 (AND)
10	100101 (or)	0001 (OR)
10	101010 (slt)	0111 (SLT)

Table 4: ALU Control Signal Generation

5 Test Program

To verify the processor's functionality, we implemented a test program that calculates the sum of an array of integers. The program consists of the following high-level steps:

1. Initialize an array with 5 elements, each with value 10
2. Store these values in memory locations starting at address 4
3. Call a subroutine to calculate the sum of the array
4. Store the sum in memory location 4

5.1 Memory Organization

Address	Initial Value	Description
0	-	Program start
4	-	First array element (later sum result)
8	-	Second array element
12	-	Third array element
16	-	Fourth array element
20	-	Fifth array element

Table 5: Memory Layout

5.2 Assembly Code

The test program is implemented as follows:

```
# Initialize registers and array
main:  addi $t0, $zero, 4    # Array base address
      addi $t1, $zero, 10   # Value to store (10)
      add  $t2, $zero, $zero # Loop counter
      addi $t3, $zero, 5    # Loop limit (5 elements)

# Store loop: store values in memory
store_loop:
      sw   $t1, 0($t0)      # Store value
      addi $t0, $t0, 4      # Increment address
      addi $t2, $t2, 1      # Increment counter
      slt  $t6, $t2, $t3    # Check if counter < limit
      beq  $t6, $zero, store_done
      j    store_loop       # Repeat

# Prepare for function call
store_done:
      addi $a0, $zero, 4    # Array base address
      addi $a1, $zero, 5    # Array size
      jal  sum_array        # Call subroutine

# Store result and exit
      sw   $v0, 4($zero)    # Store sum result at address 4
      # End program

# Subroutine to sum array elements
sum_array:
      add  $t0, $zero, $a0   # Copy array base address
      add  $t1, $zero, $a1   # Copy array size
      add  $t2, $zero, $zero # Initialize sum to 0
      add  $t3, $zero, $zero # Initialize counter to 0

# Sum loop: add each array element
sum_loop:
      lw   $t7, 0($t0)      # Load array element
      add  $t2, $t2, $t7     # Add to sum
      addi $t0, $t0, 4      # Increment address
      addi $t3, $t3, 1      # Increment counter
      slt  $t4, $t3, $t1    # Check if counter < size
      beq  $t4, $zero, sum_done
      j    sum_loop         # Repeat

# Return result
sum_done:
      add  $v0, $t2, $zero   # Return sum in $v0
      jr   $ra              # Return to caller
```

6 Simulation Results

6.1 Register Usage

Register	Name	Usage
\$0	\$zero	Constant value 0
\$4	\$a0	Argument: array base address
\$5	\$a1	Argument: array size
\$2	\$v0	Return value: sum result
\$8	\$t0	Temporary: address pointer
\$9	\$t1	Temporary: array value/size
\$10	\$t2	Temporary: counter/sum
\$11	\$t3	Temporary: counter/limit
\$12	\$t4	Temporary: comparison result
\$14	\$t6	Temporary: comparison result
\$15	\$t7	Temporary: loaded value
\$31	\$ra	Return address

Table 6: Register Usage in Test Program

6.2 Results

Time(ns)	Operation	Address/Reg	Value
25000	REG WRITE	R8	4
35000	REG WRITE	R9	20
45000	REG WRITE	R10	0
55000	REG WRITE	R11	5
65000	MEM WRITE	Addr 4	20
75000	REG WRITE	R8	8
85000	REG WRITE	R10	1
95000	REG WRITE	R14	1
125000	MEM WRITE	Addr 8	20
135000	REG WRITE	R8	12
145000	REG WRITE	R10	2
155000	REG WRITE	R14	1
185000	MEM WRITE	Addr 12	20
195000	REG WRITE	R8	16
205000	REG WRITE	R10	3
215000	REG WRITE	R14	1
245000	MEM WRITE	Addr 16	20
255000	REG WRITE	R8	20
265000	REG WRITE	R10	4
275000	REG WRITE	R14	1
305000	MEM WRITE	Addr 20	20
315000	REG WRITE	R8	24
325000	REG WRITE	R10	5
335000	REG WRITE	R14	0
355000	REG WRITE	R4	4
365000	REG WRITE	R11	5
375000	REG WRITE	R31	52
385000	REG WRITE	R8	4
395000	REG WRITE	R9	5
405000	REG WRITE	R10	0
415000	REG WRITE	R11	0
425000	REG WRITE	R15	20
435000	REG WRITE	R10	20
445000	REG WRITE	R8	8
455000	REG WRITE	R11	1
465000	REG WRITE	R12	1
495000	REG WRITE	R15	20
505000	REG WRITE	R10	40
515000	REG WRITE	R8	12
525000	REG WRITE	R11	2
535000	REG WRITE	R12	1
565000	REG WRITE	R15	20
575000	REG WRITE	R10	60
585000	REG WRITE	R8	16
595000	REG WRITE	R11	3
605000	REG WRITE	R12	1
635000	REG WRITE	R15	20
645000	REG WRITE	R10	80
655000	REG WRITE	R8	20
665000	REG WRITE	R11	4
675000	REG WRITE	R12	1
705000	REG WRITE	R15	20
715000	REG WRITE	R10	100
725000	REG WRITE	R8	24
735000	REG WRITE	R11	5
745000	REG WRITE	R12	0
765000	REG WRITE	R2	100
785000	MEM WRITE	Addr 4	100

=== Final State ===

--- Register File Contents (Non-zero only) ---

--- Final Data Memory Contents ---

Mem[0] = x
 Mem[1] = 100
 Mem[2] = 20
 Mem[3] = 20
 Mem[4] = 20
 Mem[5] = 20
 Mem[6] = x
 Mem[7] = x
 Mem[8] = x
 Mem[9] = x
 Mem[10] = x
 Mem[11] = x
 Mem[12] = x
 Mem[13] = x
 Mem[14] = x
 Mem[15] = x

=== Simulation Completed ===
 milange.v:128: \$finish called at 920000 (1ps)
 dhruvrpansuriya@Dhruvs-MacBook-Air Lab 8 %

Run Testcases 0 0 0 0

Figure 7: Simulation Results

7 Analysis of Execution

The test program successfully:

- Initialised an array of 5 elements, each with a value of 10
- Calculated the sum (50) through a subroutine call
- Stored the result back in memory

The execution demonstrates:

- Data transfer instructions (lw, sw)
- Arithmetic operations (add, addi)
- Logical operations (set-less-than)
- Control flow (branches, jumps)
- Subroutine mechanisms (jal, jr)

The trace shows that the processor correctly executed all instruction types and maintained proper program flow.

8 Implementation Challenges

During the implementation, we faced several challenges:

8.1 Control Signal Generation

Creating a comprehensive and correct control unit that generates appropriate signals for all instructions was complex. We needed to carefully map each instruction to its control signals while avoiding conflicts.

8.2 ALU Design

Implementing a gate-level ALU that supports all required operations (ADD, SUB, AND, OR, SLT) required careful attention to detail, especially for the SLT operation which needed to examine the sign bit.

8.3 Jump and Branch Logic

Calculating the correct address for branches and jumps required proper address manipulation and coordination between multiple components.

8.4 Testing and Debugging

Developing an effective test program that exercised all instructions and writing a comprehensive testbench to monitor execution were challenging but essential for verification.

9 Performance Considerations

The single-cycle implementation has the advantage of simplicity but also limitations:

9.1 Advantages

- Simple control logic as each instruction completes in one cycle
- No need for complex pipeline control or hazard detection
- Direct instruction execution makes debugging simpler

9.2 Limitations

- Clock period limited by the longest instruction path (memory access instructions)
- Higher power consumption as all units are active during each cycle
- Less efficient use of hardware resources compared to pipelined implementations

10 Conclusion

We successfully implemented a single-cycle 32-bit MIPS processor that supports the required instruction subset: load/store operations, arithmetic/logical operations, branching, and subroutine handling. The processor was implemented using a modular approach with separate components for each datapath element.

Our test program demonstrated that the processor correctly executes all instruction types and can perform meaningful computation. The simulation results verified the correct behaviour of the processor in handling memory operations, arithmetic calculations, and control flow.